

Använda namespaces

För att använda en symbol från ett namespace placerar man namnet på namespacet följt av `::` före symbolen då den används. Allmänt:

```
namespacenamn::symbolnamn
```

Vi kan alltså placera `std::` före alla klasser som vi använder som tillhör standarden för C++. Vi kunde skriva ett avancerat "Hello World"-program på följande sätt:

```
#include <iostream>

int main () {
    int Ar;

    // läs in ett år
    std::cout << "Skriv ett år: ";
    std::cin >> Ar;

    // skriv vår hälsning
    std::cout << "Hello world, detta är år " << Ar << endl;
}
```

På de flesta system behöver man inte explicit ange att man vill använda klasser och andra definitioner från namespacet `std`, vilket gäller det system som detta kompendium utvecklats på, men vissa system som bättre följer standarden för C++ kräver det. På ett sådant system kompilerar ovanstående program inte om man lämnar bort prefixet `std::` från `cin` och `cout`.

I alla program som vi hittills skrivit i detta kompendium borde vi använda oss av namespaces. Om den kompilator du använder dig av inte kompilerar exempelprogrammen kan du försöka placera `std::` före t.ex. `cin`, `cout` o.dyl, eller använda syntaxen som presenteras nedan för att alltid använda `std`. I framtida versioner av kompendiet använder kanske alla exempelprogram sig av namespaces på ett helt standardenligt sätt.

Alltid använda ett namespace

Genom att skriva ett namespacenamn följt av `::` kan vi explicit ange att vi vill ha en viss symbol från ett visst namespace, men det kan bli jobbigt i längden. Det är enkelt att glömma att placera `std::` före `cin` och `cout` varje gång de används. Man kan istället definiera att ett visst namespace alltid skall användas, d.v.s. alla symboler skall kunna användas utan att behöva skriva ett prefix före varje symbol. Detta görs med deklarationen `using namespace` som används allmänt enligt följande:

```
using namespace namespacenamn;
```

Om denna deklaration förekommer t.ex. i början på en fil kan man i den filen och alla som inkluderar denna använda symboler från *namespace*namn utan att tvingas skriva ett prefix. Deklarationen säger att vi nu framöver använder oss av allting som finns i det givna namespace. Vi kan skriva om programmet från ovan enligt:

```
#include <iostream>

using namespace std;

int main () {
    int Ar;

    // läs in ett år
    cout << "Skriv ett år: ";
    cin >> Ar;

    // skriv vår hälsning
    cout << "Hello world, detta är år " << Ar << endl;
}
```

Vi kan placera en `using`-sats i princip var som helst i en programfil där man normalt kan ha andra satser. I normala fall placerar man dock `using`-satser i början av en fil.

Man kan naturligtvis `using`-deklarerera flera olika namespaces. Detta är praktiskt om de inte innehåller namnkollisioner av något slag.

Använda subset av namespace

I alla fall kanske man inte vill "importera" alla symboler som finns i ett namespace, utan endast några få, och använda resten via ett prefix. Använder man satsen `using namespace namn`; får man ju med alla symboler i den globala symbolrymden. Man kan dock istället deklarerera att endast vissa symboler används. Allmänt skriver man då:

```
using namespace namn::symbolnamn;
```

Ett exempel kan vara att vi vill kunna använda klassen `string` från namespace `std` utan prefixet `std::`, men andra klasser bör bibehålla prefixet. Vi kan då skriva:

```
using std::string;
```

I ett sammanhang kan det se ut på följande sätt:

```
#include <iostream>
#include <string>

int main () {
    // använd string från namespace std
    using std::string;

    string Ord;

    // läs in ett år
    std::cout << "Skriv ett ord: ";
    std::cin >> Ord;

    // skriv vår hälsning
    std::cout << "Du skrev in: " << Ord << endl;
}
```

Genom att explicit välja symboler kan man även reda ut namnkollisioner. Man kan då precisera vilken version av någon viss klass som avses då klassen används.

Klasser som namespaces

Notera att då man implementerar metoder för klasser utanför själva `class`-definitionen använder man samma syntax som vid namespaces. Om vi t.ex. implementerar en metod `print()` för en klass `Image` ser det ju t.ex. ut på följande sätt:

```
void Image::print () {  
    ...  
}
```

Här säger vi explicit att vi implementerar en metod `print()` som tillhör ett namespace `Image`. Klasser fungerar alltså även som namespaces, eftersom de associerar ett antal metoder och data med ett namn (klassens namn).

[Förutgående](#)
Namespaces

[Hem](#)
[Upp](#)

[Nästa](#)
Skapa egna namespaces